AGORA — Official Feature List

Agentic Generator and OCR-based Recognition engine platform

October 14, 2026

Overview

AGORA is a web platform where teachers create, collect and check written, objective and coding assessments, plan lessons, and track class and student performance, with AI assisting and the teacher always able to review. It is built for colleges and universities: a class is a course offered in a semester, and students enroll course by course rather than in a fixed homeroom section. Its full title is *AGORA: Agentic Generator and OCR-based Recognition engine platform*.

Users: Super/Master Admins (the research team or company that administers the platform), College Admins, Teachers (faculty and instructors) and Students.

Design principles

- **Teacher in the loop.** AI scores are suggestions the teacher can review and override. Low-confidence OCR words are flagged for human review, and final grading is never fully autonomous.
- **Accounts are provisioned, not self-registered.** Colleges, college admins, teachers and students are created from the top down.
- **Cheap, deterministic checking where possible.** Objective items are checked by direct answer-key comparison, and code by the output of test cases, with no model involved.
- **Open models, no per-call billing.** The language model runs locally (Qwen 2.5 via Ollama). The three checker models are open-weight and hosted on the project's own Hugging Face Space, so there is no per-call billing and no commercial per-call API. Scanned images and answer text sent to the checker models are processed in that Space rather than on the college's own computers.

Features at a glance

#	Feature	What it does
1	Account Registration & Provisioning	Top-down creation of colleges, college admins, teacher and student accounts with role-based access
2	Online Classroom Management	Assigned classes (a course offered in a semester) where teachers manage enrolled students, assessments and class tracking
3	Sandbox: Manual Template Mapping	Teacher-defined, reusable answer-sheet layouts with answer keys
4	OCR & NLP Essay / Short-Answer Scorer	Scores handwritten (scanned) or typed written answers
5	AI Agentic Console	Generates, secures and distributes quizzes from natural-language instructions
6	ACTS NoteBook	Generates Outcome-Based Education lesson plans from lecture materials and learning goals or course outcomes
7	Digital E-Classrecord	Term-based grade ledger with weighted, automatic computation and conversion to the college grading scale
8	Class Performance Analytics	Per-class dashboards of averages, distributions and missed items, with department and program roll-ups
9	Student Retention Analytics	4-quadrant matrix of grades against attendance, with college-set thresholds, to flag at-risk students
10	Online Code Compiler / Code Checker	Runs student-submitted Python code and grades it from the output it produces (in development)
11	External API & Data Architecture	Hosted model API for outside platforms and for AGORA itself, plus the shared PostgreSQL data layer

Feature 1: Account Registration & Provisioning

Core, security-critical. Every account in AGORA is created by an authority above it, and there is no public sign-up anywhere in the system. This is the entry point for every other feature, since nothing is reachable without an account and a defined role.

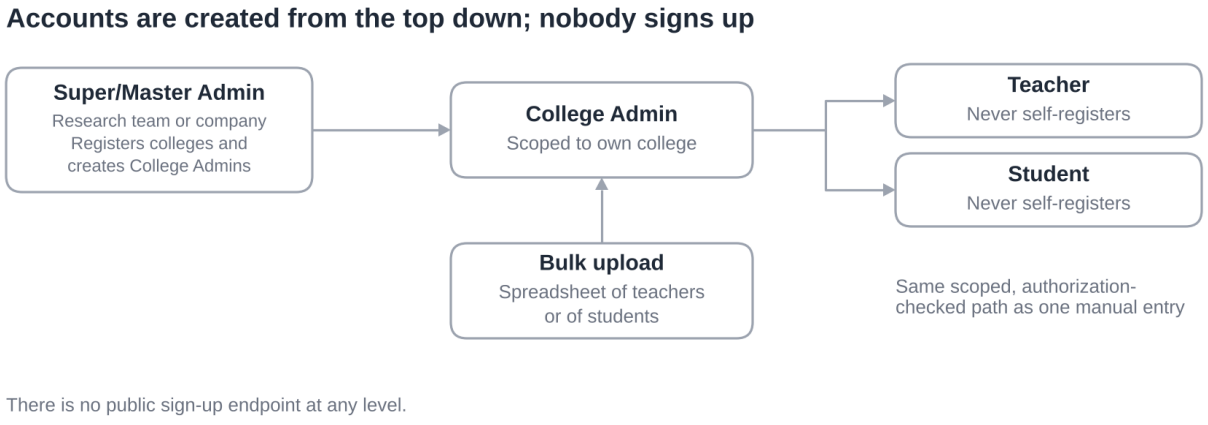


Figure 1. Provisioning hierarchy: accounts are created top-down and never self-registered.

Accounts follow a top-down hierarchy:

1. A **Super/Master Admin**, the research team or company that administers the platform, registers and manages the participating colleges and creates each college's College Admin account.
2. A **College Admin** (for example a registrar, dean or program head), scoped strictly to their own college, creates or bulk-uploads teacher and student accounts for that college. The scoping is enforced structurally, so cross-college account creation is not possible.
3. **Teachers and students** receive accounts from their College Admin and never self-register.

This mirrors how colleges onboard people in practice: the college already has the definitive list of who is employed to teach, whether full-time, part-time or visiting, and of who is enrolled. It also removes impersonation, spam registration and unverified-identity risks, since there is no open sign-up form to defend.

Core functions

- **Bulk upload.** A College Admin can onboard many teachers or many students at once from a spreadsheet (a teacher's department, rank and employment status; a student's number, degree program and year level). Bulk entries go through the same scoped, authorization-checked path as a single manual entry, and every upload is logged.
- **College records on accounts.** Teacher accounts are tied to an academic department; student accounts carry a student number, a degree program and a year level, so classes, rosters and analytics can be scoped correctly. Employee and student numbers are unique within a college.
- **Role-based access control.** Each role sees only the modules and data it is authorized for. For example, E-Classrecord and analytics are limited to administrators and the faculty assigned to the class.
- **Credential issuance.** Passwords are stored in encrypted form and must meet minimum security standards. Temporary passwords are used during development, and every new account is flagged to change its password on first login. A short-lived, single-use invite link is planned before deployment.

Technologies

- **Python (FastAPI) and Node.js** make up the application server. It creates accounts, processes bulk uploads and checks, on every request, that the person acting is allowed to act on that college.
- **A secure database (PostgreSQL)** stores the users, the four role profiles, the invite links (kept only as hashes) and a log of every bulk upload, for teachers or students, with its rows created, rows failed and errors.
- **Encrypted passwords and hashed tokens** keep passwords and invite links from being stored in readable form.
- **Design reference.** The account-creation approach follows the stars-sys-sync (jerome branch) codebase.

System limitations

Default or temporary passwords are used during development, so the single-use invite link has to be in place before deployment. A bulk upload is only as good as its spreadsheet: a wrong department or program, a duplicate employee or student number or a malformed row fails and has to be corrected and uploaded again by the College Admin. Because every account is created from above, a college cannot start using AGORA until the Super Admin has registered it and created its College Admin, and teachers and students cannot log in until that admin creates their accounts.

Feature 2: Online Classroom Management

Core. This is the workspace where a teacher spends most of the day. After the college creates the teacher's account (Feature 1), the teacher logs in and sees the classes assigned to them this semester rather than starting from an empty account.

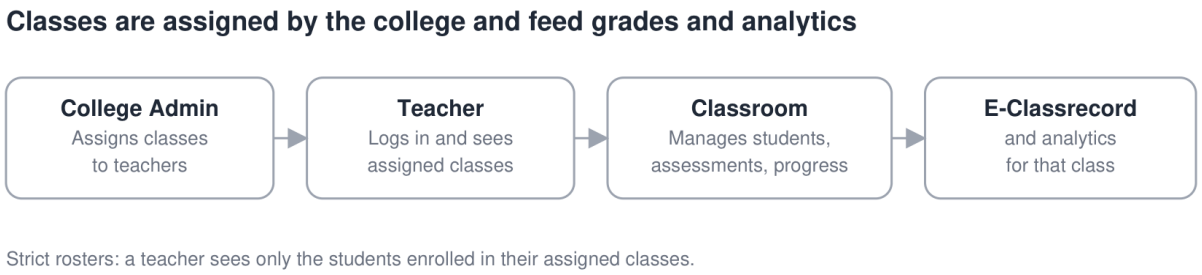


Figure 2. Classroom assignment and use.

Core functions

- **Assigned, not self-created.** Classes are assigned to teachers by the college, following the same top-down logic as account provisioning. Each class is one course from the college's catalog offered in one semester, with a section code, schedule and room. The college already knows which teacher handles which course section, so this avoids duplicate, unauthorized or inconsistent class records.
- **Strict rosters.** College students enroll course by course, so each student has a separate enrollment, and often a separate teacher, in every course they take. A teacher manages only the roster of students enrolled in their assigned classes and tracks those specific cohorts.
- **Inside a classroom,** the teacher manages enrolled students, manages the assessments given to that class, and tracks how the class is performing over time.
- **Semester view.** Teacher views and the Central Interface default to the current semester, and earlier semesters stay available for review.

- **Account and catalog management.** Administrators provision and maintain teacher and student accounts, roles and permissions, together with the academic catalog the classes are built from: departments, degree programs, courses and course learning outcomes.
- **Feeds other features.** Classroom results flow into the E-Classrecord (Feature 7) and the analytics dashboards (Features 8 and 9) for that class.
- **Central Interface.** A landing dashboard summarizes active classes, pending assessments, notifications and key system statistics for quick decisions.

Technologies

- **React** builds the Central Interface and the classroom screens, which adapt to PC, laptop, tablet and phone.
- **Python (FastAPI) and Node.js** serve the data and check that every request touches only the classes assigned to the teacher making it.
- **A secure database (PostgreSQL)** stores the academic catalog, semesters, classes, enrollments (with a status such as enrolled, dropped, withdrawn or completed) and notifications.

System limitations

Teachers cannot create their own classes, so the classroom depends on the college entering the catalog, the class assignments and the enrollments correctly and on time. A late or missing enrollment update shows up as a wrong roster, and from there as wrong grades and analytics. Each college can have only one current semester at a time.

Feature 3: Sandbox: Manual Template Mapping

Core. Teachers define their own answer-sheet layout once and reuse it for every exam that shares the format. Departments and instructors print exam sheets in different layouts, with different spacing, question order and answer regions, so a single hardcoded template cannot serve every course.

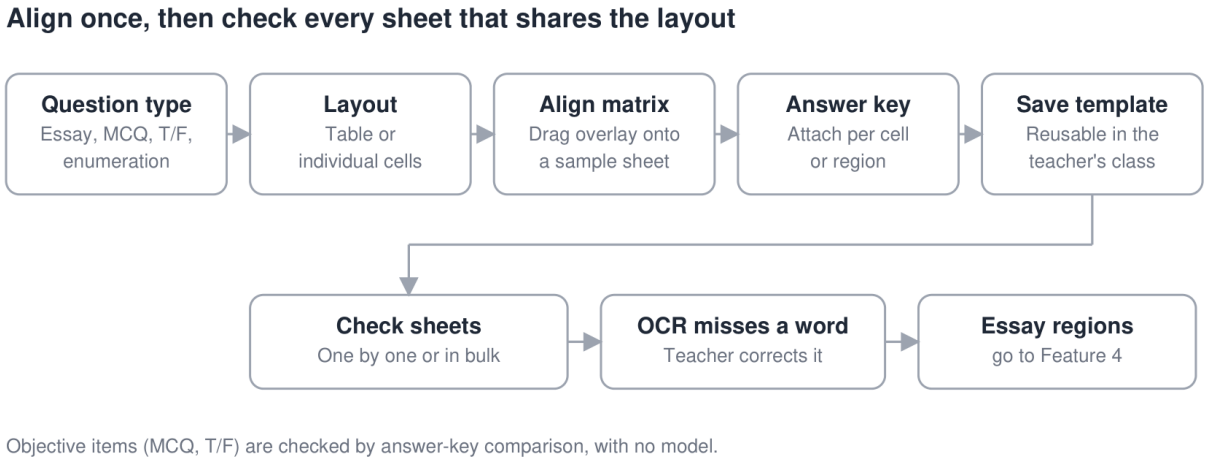


Figure 3. Template creation, reuse and checking workflow.

Core functions

Each answer-sheet layout is defined once and then reused to check every sheet that shares it.

The one-time setup, per sheet layout:

1. Choose the question type for a region of the sheet: Essay, Multiple Choice, True/False, Enumeration and so on, since each type is read and checked differently.
2. Choose the layout for grid-style items: a table or individual cells.
3. Align the matrix. The teacher drags a matrix overlay on a sample sheet image until it lines up with the printed answer positions.
4. Attach the answer key per cell or region. This lets objective items (MCQ, T/F) be checked by simple answer-key comparison rather than a model.
5. Save as a reusable template within the teacher's classroom. Because a teacher often handles several sections of the same course, a saved template can be duplicated to their other classes for that course.

Once a template exists, checking is repeatable:

- **One-by-one or bulk.** Sheets can be checked individually or processed in bulk.
- **Self-correction.** If OCR fails to recognize a word, the teacher corrects it directly in the interface rather than the system silently guessing or discarding the answer.
- **Scanned sheets feed the scorer.** Essay and short-answer regions go to the OCR & NLP scorer (Feature 4), and the results flow to the E-Classrecord (Feature 7).

Technologies

- **React** provides the alignment screen, where the teacher drags the matrix overlay onto a sample sheet image.
- **Python (FastAPI) and Node.js** read the saved geometry of each region and apply it to every scanned sheet in a batch.
- **A relational database (PostgreSQL)** stores the templates, their regions with the aligned geometry, and the answer key for each item.
- **Design reference.** The approach to template definition, storage and alignment draws on the starscan (feature/with-quizzes) codebase, which solves a similar calibration problem for optical mark recognition sheets.

System limitations

Alignment is manual, so every new sheet layout costs the teacher a setup step, and a matrix that is slightly off will misread every sheet that uses it. Results also depend on the scan or photo: a skewed, blurry or poorly lit sheet can shift the answer regions away from the saved template. The answer key is trusted as entered, so a wrong key gives wrong objective scores for the whole batch, and the teacher should check it before processing sheets in bulk.

Feature 4: OCR & NLP Essay / Short-Answer Scorer

Core, highlighted AI feature. This feature automatically scores student-written responses, essays and short answers, and is what makes AGORA AI-driven rather than a digitized version of manual checking. Answers can arrive in two ways, and both end at the same essay checker so the scoring logic is built once.

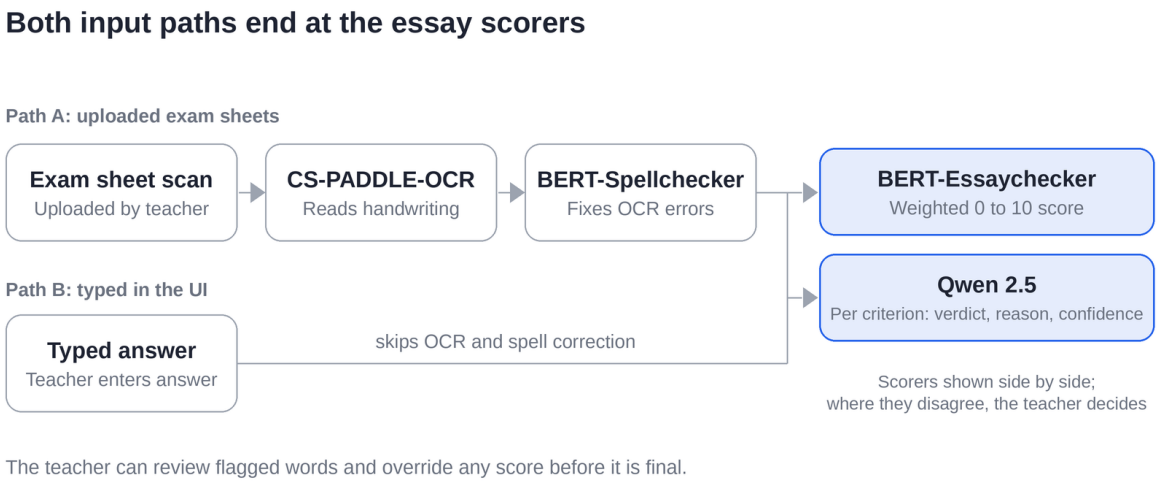


Figure 4. Dual-path scoring pipeline: uploaded sheets and typed answers end at both essay scorers.

Path A: uploaded exam sheets. Many colleges still run paper exams, so the teacher uploads the scanned or photographed answer sheets, typically using a template from Feature 3.

1 **CS-PADDLE-OCR** extracts the handwritten text.

2 **BERT-Spellchecker** corrects OCR misreads before scoring.

3 **BERT-Essaychecker** scores the corrected text.

Path B: manual typing in the UI. The teacher types the student's answer into the interface. This skips OCR and spell correction entirely and goes straight to the essay checker.

The three models

Model	Role	Training
CS-PADDLE-OCR	Reads handwriting from the scanned region. Includes page clean-up, reading-order correction and a confidence score for each recognized region, used to flag uncertain text for teacher review.	PP-OCRv6 medium recognizer fine-tuned (post-trained) on the IAM Handwriting Database and the team's own handwriting
BERT-Spellchecker	Two-stage OCR text correction: SymSpell fixes spelling and merged or split words, then BERT reranks candidates by sentence context.	BERT fine-tuned on the TU Darmstadt argumentative essays corpus
BERT-Essaychecker	Produces the final score from two sub-scores: writing quality (mechanics) and coverage of the teacher's rubric (content).	DistilBERT mechanics model fine-tuned on the Hewlett Foundation (ASAP) dataset

How the score is built

- **Mechanics sub-score:** a fine-tuned DistilBERT regression model rates writing quality and fluency on a 0 to 1 scale.
- **Content sub-score:** each rubric criterion is compared with the answer's sentences using local sentence embeddings, so paraphrases still match. Criteria are marked met, partial or not met against point values.
- **Teacher-defined rubric:** the teacher supplements the rubric in the UI, writing the criteria and point values for each question.

- **Final score:** a weighted combination of the two sub-scores, 40% mechanics and 60% content by default, shown on a 0 to 10 scale. The weights are adjustable and are stored with each assessment, together with the values actually used for every answer, so older scores can be reproduced.
- **From 0 to 10 to question points:** the 0 to 10 score is converted to the question's own points: points earned = final score ÷ 10 × the question's points. A 15-point essay scored 8 out of 10 earns 12 points.
- **Short answers** go through the same pipeline and are scored mainly against the teacher's rubric.

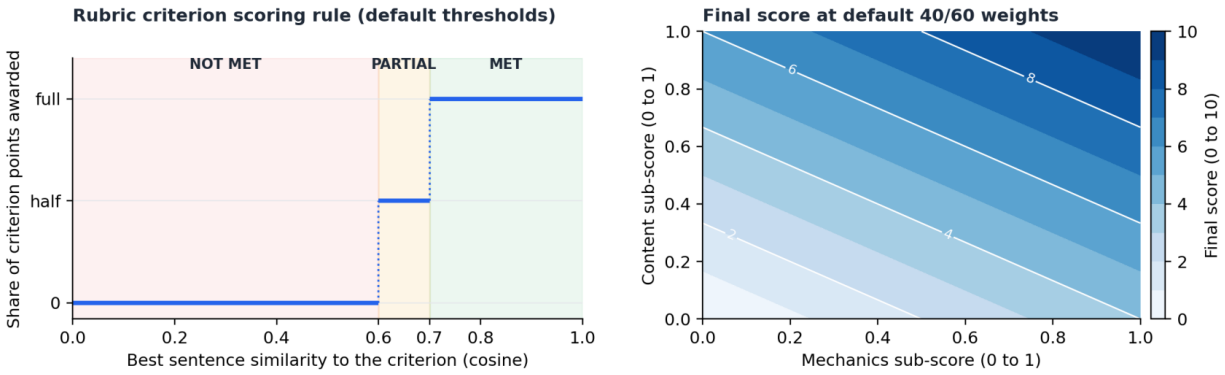


Figure 5. Left: rubric criterion scoring rule. Right: final score as a function of the two sub-scores. Both show the essay checker's default parameters.

Two scorers, one teacher review. BERT-Essaychecker is one of two essay scorers in AGORA. In the Agentic Console (Feature 5), Qwen 2.5 also scores each rubric criterion and gives a verdict, a short reason and a confidence. The weighted 0 to 10 score above is the predicted score. Qwen's per-criterion results are stored under their own label and shown beside it as a second scorer, and where the two disagree the teacher decides.

Teacher review. The predicted score is always reviewable. The teacher can correct OCR text, check flagged low-confidence words and override any score before it becomes final. Confirmed scores flow to the E-Classrecord (Feature 7).

The full scoring pipeline runs on open models with no paid API calls.

Core functions

- **Two input paths, one checker.** Scanned sheets go through OCR and spelling correction before scoring, while answers typed by the teacher go straight to the scorer.
- **Handwriting reading.** CS-PADDLE-OCR reads the handwritten regions, puts the words back into reading order line by line and gives a confidence score for each recognized region.
- **OCR correction.** BERT-Spellchecker repairs misread words and wrongly split or merged words, and leaves alone the subject terms and acronyms in its custom vocabulary list.
- **Two-part scoring.** Writing quality and rubric coverage are scored separately and combined into one final score out of 10, with 40% mechanics and 60% content by default.
- **Rubric matching.** Each rubric criterion is matched to the best-fitting sentence of the answer, so a paraphrase can still earn credit. A similarity of 0.70 or higher earns full points, 0.60 or higher earns half, and anything lower earns none. Both cutoffs are adjustable per assessment.
- **Teacher review.** The teacher can correct OCR text, check the specific flagged low-confidence words and override any score before it is final.

Technologies

- **PaddleOCR (CS-PADDLE-OCR)** uses the PP-OCRv6 medium text recognizer, fine-tuned for handwriting. Photos are scaled down to 2,000 pixels on the longest side before reading, and the recognizer is scored with character error rate and word error rate.
- **SymSpell and BERT (BERT-Spellchecker)** combine a word-frequency spelling corrector (an 82,765-word English dictionary, up to three edits per word) with bert-base-uncased, which picks the candidate that best fits the sentence. BERT was trained further on the TU Darmstadt corpus of 402 argumentative essays.
- **DistilBERT and sentence embeddings (BERT-Essaychecker)** use a fine-tuned DistilBERT regression model for writing quality and the all-MiniLM-L6-v2 sentence-embedding model for rubric matching, run through the sentence-transformers library.
- **Open models on Hugging Face Spaces.** No paid per-call API is called anywhere in the scoring pipeline. The three models are open-weight and hosted on the project's own Hugging Face Space, which AGORA's application server calls (Feature 11), and the model files are stored with Git LFS.

System limitations

The reader expects flat, upright photos, because the page-orientation and unwarping steps are switched off. Reading a full-size phone photo takes minutes on an ordinary CPU, such as the free hosting tier's, which is why photos are scaled down, so scanning many long answers goes faster on stronger hardware. Because the models are hosted, scanned images and answer text leave the college's computers for the Space, and a free-tier Space can be slow to start.

The mechanics model reads the first 512 tokens of an answer, so a very long answer is judged on its opening part. AI scores are suggestions: the teacher reviews them and can override any score before it becomes final.

Feature 5: AI Agentic Console

Core, AI-driven. The Agentic Console is the main AI-powered assistant of AGORA, built for creating quizzes, sending them to students, and checking how a class is performing. Instead of filling in forms, a teacher types what they need in everyday language, such as creating a quiz, sending a quiz to a class, or asking how a class is doing, and a team of six specialized agents carries the request out step by step. The teacher always has the final say: every quiz is previewed and approved before it is released, and any essay scored by the AI must be confirmed by the teacher before the score becomes official.

Supported tasks

- **Quiz generation.** The teacher asks for a quiz based on a topic, learning competencies or an uploaded file. For example: "Make a 20-item quiz on process scheduling, 15 multiple choice and 5 true/false, with one essay."
- **Quiz dissemination.** The teacher sends a quiz to the students enrolled in a class, either automatically after a newly created quiz is approved, or on request for a quiz saved earlier. For example: "Send Quiz 2 to CS 301-A, due next Friday at 5 PM."
- **Class performance retrieval.** The teacher asks about a class and the system answers from the records already stored in it: class averages, the results of a specific quiz, a student's standing, or the list of students flagged At Risk. For example: "Who failed Quiz 2 in CS 301-A?" or "Show the Midterm average of my CS 301-B section." The assessment details and data of the chosen class are presented in a clear, readable way.



Figure 6. The six-node LangGraph pipeline of the Agentic Console, including the teacher review stop and the pending essay scores.

Core functions

- **Understanding everyday language.** The system turns the teacher's plain message into the details it needs: the type of task, the class, the topic, the difficulty, the number of items, the kinds of questions and the deadline. The AI only picks out these details from the message. It never searches the database by itself.
- **Sending requests the right way.** After checking the request, the system directs it down the correct path: creating a quiz, only sending a quiz, or retrieving class information.
- **Remembering and correcting itself.** The system keeps track of each request from start to finish. If a step makes a mistake, for example if the quiz comes out in the wrong format, it goes back, fixes the problem and tries again a limited number of times, without forgetting what the teacher originally asked for.
- **Doing tasks on its own.** The agents also carry out actions in the system: understanding dates, finding uploaded documents, checking that the quiz is in the right format, creating secure links, sending emails, saving grades and looking up class information.
- **Teacher review before release.** After a quiz is created, the system pauses so the teacher can read it, edit it and approve it. If an essay question has no scoring guide (rubric), the teacher adds it at this stage. Nothing is sent to students until the teacher approves.
- **Secure quiz links.** Each quiz is sent through a link that has an expiry time and cannot be changed or faked, emailed to the students in the class.
- **Access only to your own classes.** Every action is limited to the classes assigned to the teacher making the request. The console cannot look up, send or change anything for another teacher's class.

The six-step agent team

Step	Agent	Role
1	Validator	The gatekeeper. Checks that the request is a valid school-related task, refuses unrelated or unsupported requests, and decides whether the teacher wants to create a quiz, send a quiz or look up class information.
2	Completer	The detail collector. Gathers what the task needs (task, class, topic, difficulty, number of items, question types, grading period and learning goals) and turns phrases like "due next Friday at 5 PM" into an exact date and time. If something is missing, including a rubric for an essay question, it asks the teacher.
3	Creator	The quiz writer, used only when a new quiz is made. Qwen 2.5 writes the questions, the wrong-answer choices and the answer keys, reading the uploaded file first if the quiz is based on one. The output is checked for the correct format, and the Creator tries again if it is not.
Stop	Teacher review checkpoint	The approval stop, for new quizzes only. The system pauses so the teacher can preview, edit and approve the quiz and add essay scoring guides, and continues only after approval. A quiz that was already approved and saved earlier skips this stop and goes straight to confirming the class and the deadline.
4	Distributor	The sender, used when creating and sending a quiz or when only sending a saved one. Creates the secure link with an expiry time, then emails each student in the class the link, the deadline and instructions on how to submit.
5	Checker	The grader. Starts when a student submits an answer sheet or when the deadline arrives. Multiple choice, true/false and enumeration are compared with the answer key, and code is graded by running it against test cases (Feature 10). Essay and short-answer questions are scored against the teacher's rubric one criterion at a time by both essay scorers: BERT-Essaychecker (Feature 4) and Qwen 2.5, which gives a verdict, a short reason and a confidence for each criterion. AI essay scores are marked "pending" until the teacher confirms them.
6	Recorder	The record keeper. Saves the checked scores into the E-Classrecord. If the quiz has no column in the grid yet, it creates one in the quiz's grading period and component, then writes each student's score. Quizzes that include essays are not recorded right away, because the teacher must first double-check the AI's essay scores.

Sending quizzes to students

Once the Distributor has prepared the quiz, the system automatically emails each enrolled student through Gmail SMTP. The email contains only the secure link, the deadline and instructions for submitting. Scores and results are never sent by email. Students see their approved results inside the system.

Technologies

- **LangGraph** is the workflow system. It coordinates the six steps, passes information from one step to the next, chooses the correct path for each request and handles retries. It saves its progress, so the system can pause for the teacher's approval and continue afterward, and so the screen can show live updates.
- **Qwen 2.5 via Ollama** is the locally run AI model. It runs on the college's own computers instead of an outside service, understands requests, writes quizzes, scores essays and summarizes class information. This keeps student data private and avoids usage fees.
- **Hosted checker models.** The Checker calls the BERT-Essaychecker on the project's Hugging Face Space (Feature 11) alongside Qwen 2.5.

- **Python (FastAPI)** is the system's engine. It receives the teacher's requests, runs the steps in the background and sends live progress to the screen so the page stays responsive.
- **HMAC-SHA256** protects the quiz links with a secure code that gives them an expiry time and prevents tampering. The system also checks that the teacher actually owns the class for every request.
- **React** provides the chat-style screen where teachers type requests, watch each step happen live, preview the quiz, see the class information and confirm the AI's essay scores.

Console runs are stored with the teacher's request, model settings and agent outputs.

System limitations

Running the AI model on the system's own server needs strong hardware, so the console may respond more slowly when several teachers use it at the same time. AI scoring of essays is helpful but not perfect, especially for unusual or very abstract answers, so these scores stay pending until the teacher confirms or changes them. Class information is only as current as the records in the system: if scores or attendance have not been entered yet, the console tells the teacher that the information may be outdated or incomplete. The console handles only the tasks listed above and politely declines everything else. Sending emails requires a stable internet connection to reach the email service (Gmail).

Feature 6: ACTS NoteBook (Learning Plan Generator)

Core, AI-driven. The ACTS NoteBook helps teachers prepare lesson plans without writing them from scratch. A teacher uploads teaching materials, such as lecture slides, along with the weekly learning goals. The NoteBook reads both, matches the lesson content to what students are expected to learn, and puts together a complete, ready-to-use lesson plan following the Outcome-Based Education (OBE) approach. The teacher needs no technical knowledge and does not need to know how to instruct an AI, because the system handles that part automatically. The plan appears on screen as it is being written, so the teacher can follow along, make edits and download the final version. In a college setting the goals usually come from the course's learning outcomes in its syllabus.

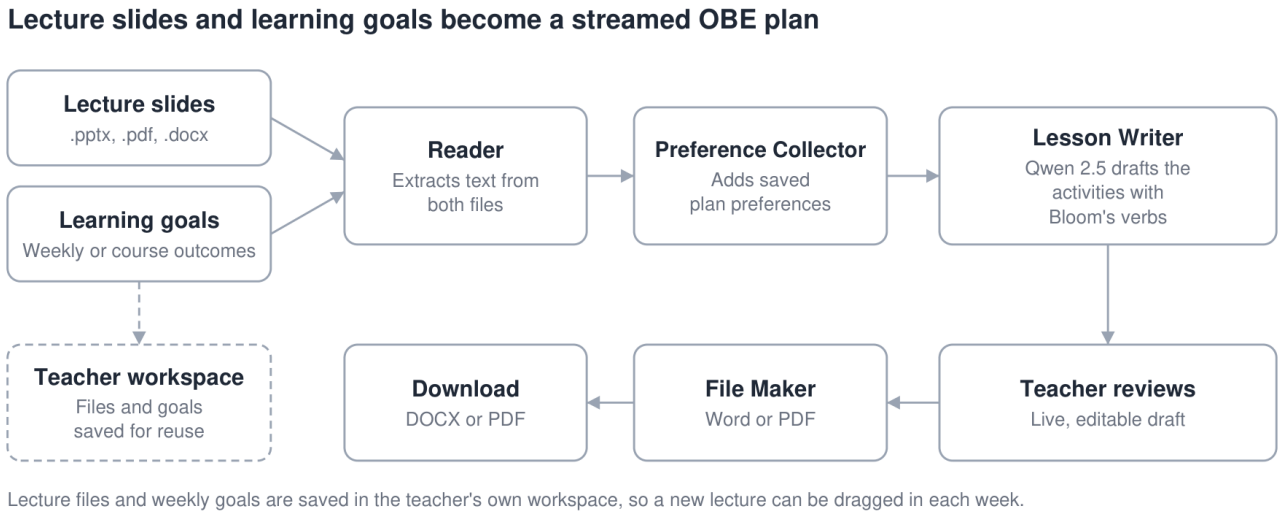


Figure 7. ACTS NoteBook processing flow.

Core functions

- **Reading two kinds of documents at once.** The NoteBook reads the teacher's lecture slides and the weekly learning goals together and pulls out the text from both so it can work with them side by side.

- **Keeping track of the plan as it is built.** The plan is built in steps. Each step can see what the previous steps produced, follow the teacher's requirements and add its own part, so the plan grows in an organized way from start to finish.
- **Matching content to learning goals.** The NoteBook connects the lecture content to each learning goal using the action words of Bloom's Taxonomy (such as explain, apply or evaluate), so each goal clearly states what students should be able to do.
- **Course outcomes.** Learning outcomes saved with the course in the academic catalog (for example CLO1 to CLO5) can be loaded as the learning goals, so a teacher does not retype them for each class or week.
- **Automatic file download.** Once the plan is ready, it is converted into a Microsoft Word (.docx) or PDF file the teacher can download, print or share.

How it works

The NoteBook works like a small team, where each member has one job.

Step	Role
The Reader	The first step. Opens the teacher's files (PowerPoint, PDF or Word) and extracts the text of the lecture and the learning goals.
The Preference Collector	Quietly looks up the teacher's saved learning plan preferences, so the plan follows their usual style without repeating it every time.
The Lesson Writer	The main step. Uses AI to study how the lecture connects to the learning goals and drafts the learning activities in a logical order.
The File Maker	The last step. Once the teacher approves the plan on screen, it prepares the Word or PDF file for download.

Bloom's Taxonomy

Bloom's Taxonomy is a hierarchical framework used in education to classify learning objectives by their level of complexity and specificity. It serves as the backbone of Outcome-Based Education because it provides a structured vocabulary of specific action verbs that dictate exactly how a student must demonstrate their knowledge.

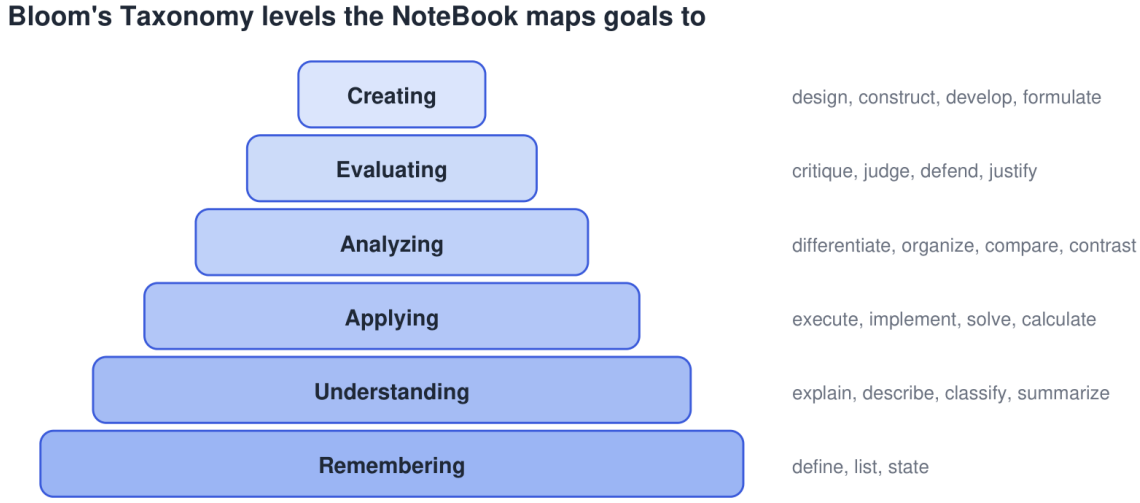


Figure 8. Bloom's Taxonomy levels and action verbs used to align goals.

Technologies

- **LangGraph** coordinates the steps and passes information from one to the next. It also lets the teacher watch the plan appear on screen as it is written.
- **Qwen 2.5 via Ollama** is the locally run AI model. It runs on the college's own computers instead of an outside service, so the college's lesson materials stay private.
- **Python (FastAPI) with WebSockets** sends progress to the teacher's screen as the work happens, so the page does not freeze or time out while large files are processed.
- **React** provides a split-screen workspace: the materials on one side and the lesson plan being written, line by line, on the other.

What the teacher gets

- **A live, editable learning plan.** The plan appears on screen as it is written, and the teacher can edit it right away.
- **Word and PDF downloads.** Finished, well-formatted files ready for use.
- **A text check.** A count of the characters extracted from each uploaded file, so the teacher can confirm the system read the materials properly.

Security and limitations

To protect the college's curriculum, the NoteBook runs on the system's own server and is accessible only to the teacher who owns the files. Uploaded lecture files and weekly learning goals are saved in the teacher's own workspace so they can be reused later. For example, a teacher can keep the same weekly goals and simply drag in a new lecture each week.

The NoteBook has two main limitations. First, the quality of the learning plan depends on the quality of the uploaded materials: unclear or incomplete files lead to weaker results. Second, running the AI model on local computers requires strong hardware, especially when many tasks run at the same time.

Feature 7: Digital E-Classrecord

Core. The E-Classrecord is the digital class record of AGORA, where teachers keep and manage their students' grades. It replaces paper and manual spreadsheets, which are easy to get wrong, with an organized record that calculates grades automatically. It receives scores from quizzes given through the Agentic Console and from scanned answer sheets checked by the OCR-based assessment checker, and lets teachers enter scores for activities done offline, so one record shows each student's progress across the whole semester.

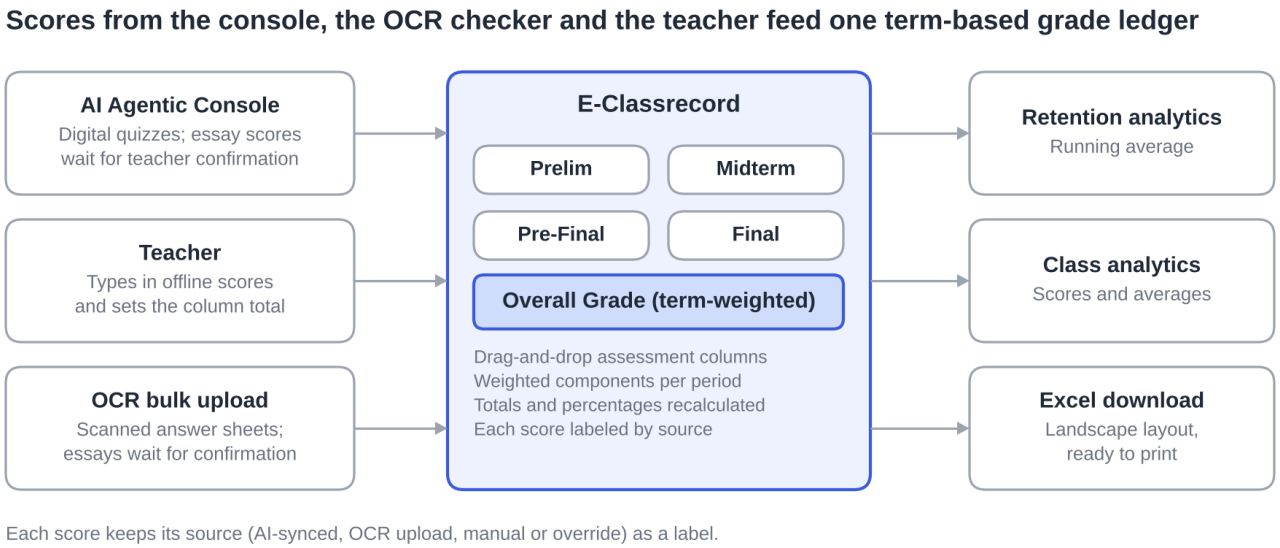


Figure 9. Data sources and outputs of the E-Classrecord.

Core functions

- **Separate tabs for each grading period.** The semester is divided into tabs for Prelim, Midterm, Pre-Final and Final, plus an Overall Grade tab that combines them. Each period stays on its own page, so the screen is not cluttered and the computations do not get mixed up.
- **Drag-and-drop columns.** Teachers add and rearrange columns for quizzes, recitations, major exams and other activities by dragging them into place on the grid. Every column belongs to a weighted grade component of its period, such as Written Work or Performance Task.
- **Weighted components and terms.** Each class sets the weight of its components per grading period (for example Written Work 40% and Performance Task 60%), and the weights of a period add up to 100. A student's grade for a period is the weighted total of that period's component percentages. The Overall Grade combines the four periods using term weights, 25% each by default and adjustable, and the four also add up to 100.
- **Automatic score entry.** When a student submits a quiz given through the Agentic Console, or scanned answer sheets are processed through the OCR checker, the scores are placed in the record automatically. A quiz that has no column yet gets one in its grading period and component. Scores from AI-checked essays, whether typed or scanned, are placed only after the teacher confirms them.
- **Automatic computation.** The record calculates column totals, grading period percentages and the running average on its own, so teachers do not need to write spreadsheet formulas.
- **College grading scale.** The Overall Grade percentage is converted to the college's own scale (for example 1.00 to 5.00, a 4.0 grade-point scale or letter grades) using bands that the College Admin configures, with passing and failing bands marked.
- **Excel download.** Teachers can export the class record as an Excel file in landscape layout, ready for printing or submission.

How it works

- **One period at a time.** When a teacher picks a tab such as Midterm, the screen shows only the activities and grades for that period.
- **Drag and save.** When a teacher drags a new column into a period, the system saves it right away and links that activity to that period and to one of its components.

- **Filling in the scores.** Cells fill in automatically for digital quizzes and for scanned answer sheets uploaded in bulk through the OCR checker, or the teacher types the scores in for offline activities.
- **Feeding the early-warning feature.** The record sends each student's latest running average to Retention Analytics (Feature 9), which uses it to show which students may need help.

Technologies

- **A drag-and-drop screen (React)** builds the grade table and makes moving columns smooth and easy.
- **A calculation engine (FastAPI)** recomputes the class averages whenever a column is added or a score is changed.
- **A secure database (PostgreSQL)** stores the grades safely, so nothing is lost even if the browser closes in the middle of an edit.

What the teacher gets

- **Grade tables per period.** A clear view of each student's scores and percentages for the selected grading period.
- **An Overall Grade sheet.** One summary that combines all four grading periods into each student's final standing, with its equivalent on the college grading scale.
- **Data for Retention Analytics.** The percentages used to show whether a student is on track or at risk.
- **An Excel copy.** A landscape-format file for printing or submission.

Security and limitations

Teachers can only open and edit the records of the classes assigned to them. Scores recorded automatically are labeled by source, so it is always clear which scores came from the system, which came from the OCR checker and which were entered or changed by the teacher.

There are two main limitations. First, a large class with many columns (for example, 50 students and 20 activities) may run slowly on older computers or browsers. Second, for activities done on paper, the teacher must enter the correct total score when creating the column. If the total is wrong, the percentages will be wrong too.

Feature 8: Class Performance Analytics

Core, statistics-based. This feature saves teachers from tallying and interpreting scores by hand. Once individual results exist, from objective answer-key checking, the essay scorer or the E-Classrecord, they are consolidated into a per-class dashboard. It runs on standard statistics, so its value does not depend on a machine learning model.

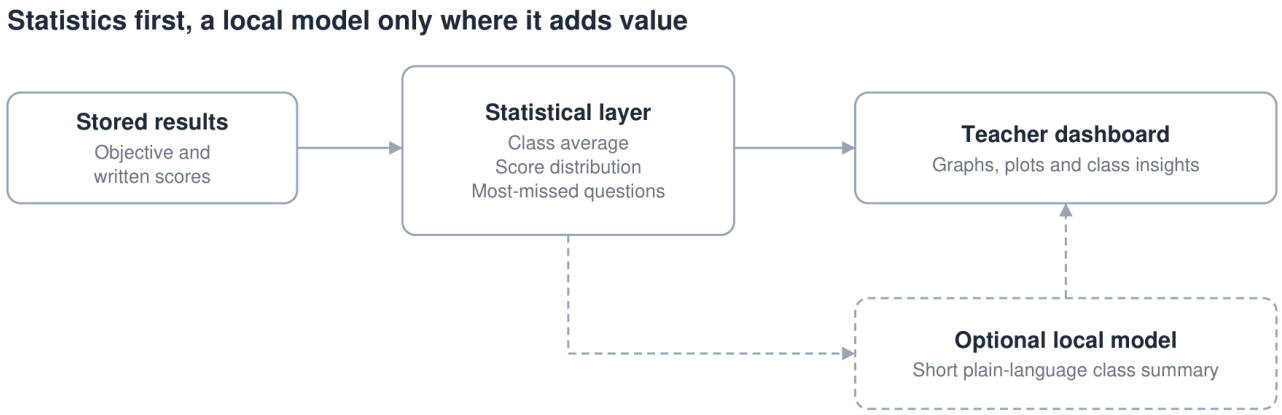


Figure 10. Analytics pipeline: statistics first, optional local model.

Core functions

- **Per-class dashboard.** Graphs, plots and summary statistics drawn from stored results, covering both objective items and written responses.
- **Baseline metrics.** Class average, score distribution curves, the questions most commonly missed, and how a class compares across exams and activities over time.
- **College-wide view.** Administrators see aggregated performance and system usage across departments, programs and class sections, and can compare the sections of one course taught by different teachers.
- **Optional language-model summary.** A local model can add a short plain-language summary of class performance. It follows the free, locally run pattern of the CORNELIA reference application, so no paid per-call API is introduced.

Technologies

- **A statistical layer (Python, FastAPI)** computes the class average, median, spread, extremes and per-question miss rates. None of it depends on a machine learning model.
- **A chart display (React with a charting library)** draws the graphs on the teacher and administrator dashboards.
- **A secure database (PostgreSQL)** stores the results the statistics are computed from and the saved class performance snapshots.
- **Qwen 2.5 via Ollama (optional)** writes the short plain-language summary on the college's own computers, so no paid per-call API is added.

System limitations

The dashboards are only as complete as the stored results: scores that have not been recorded, or essay scores still pending teacher confirmation, are missing from the numbers. Statistics describe what happened but not why, so the teacher still has to interpret them, and the average and distribution of a small class change a lot with a single score. The optional summary needs the local model and strong hardware, so it can respond more slowly when several teachers use it at the same time, and it is a convenience for reading the class, not a record of it.

Feature 9: Student Retention Analytics

Core, rule-based. Retention Analytics is the early-warning feature of AGORA. It uses the grades in the E-Classrecord and the attendance recorded by teachers to show which students may be falling behind. Each student is given one of three labels (Safe, For Counseling or At Risk) and placed on a simple chart, so teachers and administrators can see who needs help and why, well before the end of the term.

Two inputs, one 75% rule, four zones, three labels

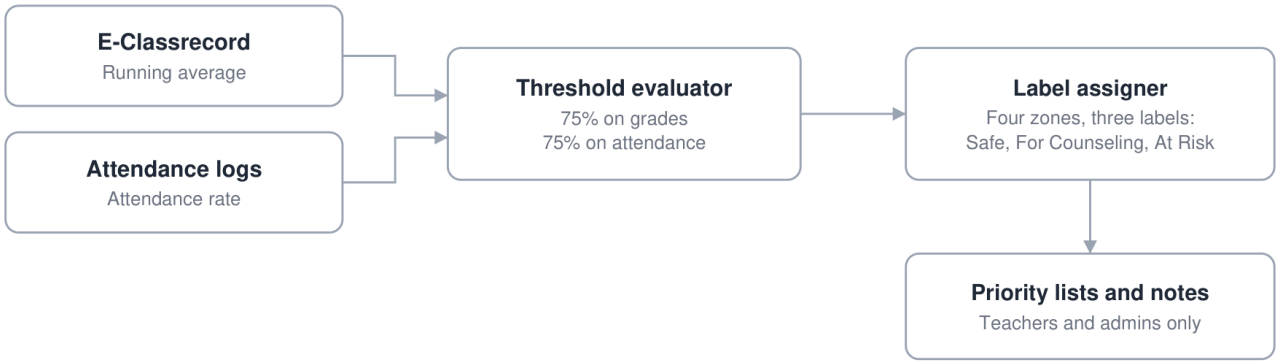


Figure 11. Retention analytics data flow. The 75% baseline is the default and can be set per college.

The feature uses clear, fixed rules instead of a "black box" AI, so teachers and counselors can easily explain why a student was labeled. Each student's running average (from the E-Classrecord) and attendance percentage (from the teacher's attendance records) in a class are compared with the standard for both academics and attendance, 75% by default. Because passing marks and attendance rules differ between colleges, each college can set its own baselines.

Grades and attendance against a 75% line give four zones and three labels

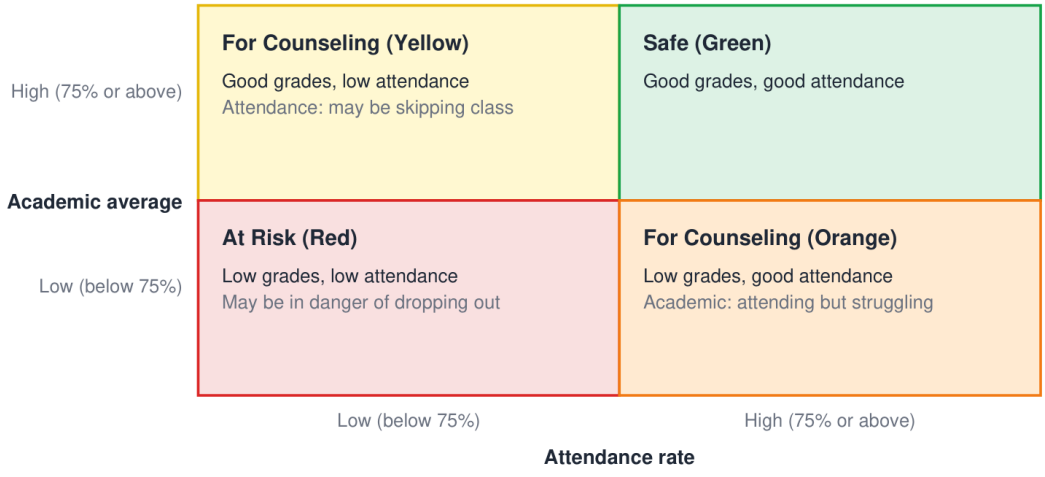


Figure 12. Retention matrix: four zones and three labels from a 75% threshold on grades and attendance (the default; configurable per college).

Core functions

- **Four-zone student chart.** Students are plotted on a chart divided into four color-coded zones based on two things: their grades and their attendance. Each student is given one of three labels: Safe, For Counseling or At Risk.

- **Suggested actions.** The system suggests what to do depending on the reason a student is labeled. A student labeled For Counseling because of low grades may need tutoring, while one labeled For Counseling because of low attendance may need attendance counseling. An At Risk student is recommended for referral to the guidance counselor.
- **Per-course classification.** A student is classified in each class they are enrolled in, so someone can be safe in one course and at risk in another.
- **Automatic updates.** A student's label changes as soon as a new quiz score is recorded or the teacher logs an absence.
- **College-wide watchlists.** The College Admin can see the results of all classes in one view, filterable by department and program, which helps identify courses and programs that need more support.

How it works

- **Collecting the data.** The system gathers each student's running average from the E-Classrecord and attendance percentage from the teacher's attendance records.
- **Checking against the standard.** Both values are compared with the standard of 75% (or the college's own baseline).
- **Assigning the label.** Based on the results, each student is placed in one of four zones and given one of three labels:
  - **Safe (Green):** good grades and good attendance.
  - **For Counseling, Attendance (Yellow):** good grades but low attendance. The student may be skipping class.
  - **For Counseling, Academic (Orange):** low grades but good attendance. The student is attending but struggling.
  - **At Risk (Red):** low grades and low attendance. The student may be in danger of dropping out.

The update process

The feature runs in the background and updates by itself as the term goes on.

1. **Data comes in.** Quiz scores recorded by the system, scores entered by the teacher and attendance logs are collected.
2. **Percentages are recalculated.** The system updates each student's grade and attendance percentages and compares them with the 75% standard.
3. **Labels are updated.** A student's label changes when the numbers change. For example, a student moves from Safe to For Counseling when attendance falls below 75%.
4. **Lists are refreshed.** The chart updates, and At Risk (Red) students appear at the top of the teacher's priority list.

Technologies

- **A calculation engine (FastAPI)** computes the percentages, decides each student's label and prepares the results for the screen.
- **A chart display (React with a charting library)** shows the chart and color-coded labels on the teacher and administrator dashboards.
- **A secure database (PostgreSQL)** stores the grades and attendance records the feature relies on.

What the teacher gets

- **A class chart.** Every student is plotted on one chart, so the teacher can see how the whole class is doing at a glance.
- **Priority lists.** Filterable lists show At Risk (Red) students first, followed by For Counseling (Orange and Yellow) students, so the teacher knows who to help first.
- **Action notes.** Suggestions added to a student's profile, such as "Refer to Guidance Counselor" or "Provide remedial tutoring."

Security and limitations

Because this information is sensitive, teachers can only see the results of students enrolled in the classes assigned to them. The labels, colors and suggested actions are visible only to teachers and administrators. They are hidden from students, to avoid discouraging them or exposing private information.

The main limitation is that the chart relies only on numbers: grades and attendance. It cannot detect other things that affect whether a student stays in school, such as health or emotional problems, changes at home or bullying. Its accuracy also depends on teachers: if attendance is not logged on time or offline scores are encoded late, the chart will be out of date and may miss a student who needs help.

Feature 10: Online Code Compiler / Code Checker

Proposed, prototype built, grading in development. This feature lets AGORA check coding assessments, which would otherwise fall outside a checker built around text and objective items. Since AGORA targets colleges, including Computer Science and IT programs, it opens the system to a large share of real coursework.

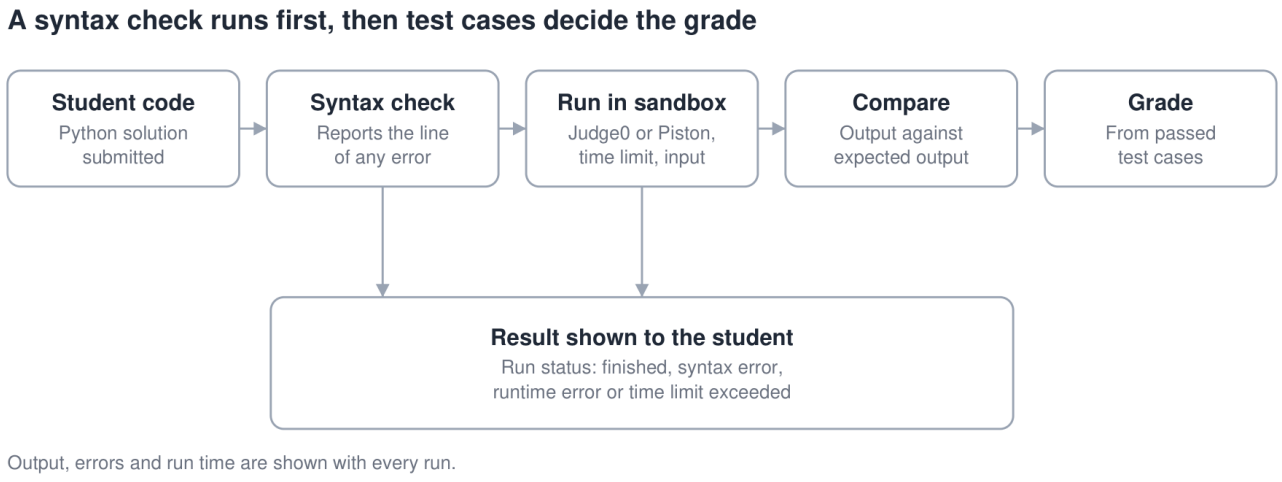


Figure 13. Code checker execution and grading flow.

Core functions

- **Code submission.** Students submit code for a problem. Python is the supported language, which keeps the execution setup and grading logic simple.
- **Compile check.** A syntax check runs first and reports the exact line number and message of any syntax error.
- **Execution with input.** The code runs with optional program input (values read by input()), under a time limit (5 seconds in the prototype), with a cap on returned output (20,000 characters).

- **Clear results.** The student sees the output, any errors, the run status (finished, syntax error, runtime error or time limit exceeded) and the run time.
- **Grading by output (in development).** Submitted code is executed against a defined set of test cases (input and expected-output pairs) attached to each problem. The output of each run is compared with the expected output, every passed test case earns its points, and the total becomes the answer's score. The teacher can review and override it like any other score before it reaches the E-Classrecord.

Execution engine. Code is run through an established, self-hosted, open-source sandbox engine, Judge0 or Piston. Both are free software with a track record in online-judge platforms and provide resource limits, timeouts and isolation from the host system.

Prototype. A working Flask prototype demonstrates the editor, syntax check, input handling, output and error display, and run timing, with a switchable runner so it can call a self-hosted Piston server.

Technologies

- **Flask** runs the prototype: the code editor, the input box, the output and error panels and the run timer. The finished feature is planned to be served through the same FastAPI application server as the rest of AGORA.
- **Python's compile step** checks the syntax first and reports the line number of any error.
- **Judge0 or Piston** is the planned sandbox engine: a self-hosted, open-source service that runs student code with time limits, resource limits and isolation from the host. The prototype can call a self-hosted Piston server.
- **A local runner** is the prototype's default. It runs the code in a separate Python process with a 5-second time limit, a 256 MB memory limit where the operating system supports it, and a cap of 20,000 characters on returned output.
- **A relational database (PostgreSQL)** stores the coding problems, their test cases and the status, run time, passed cases and per-case results of each run.

System limitations

This feature is proposed and still in development, and only a prototype exists. The prototype checks syntax, runs code with input and shows the output, errors, status and run time. Grading from the output is the part being built: comparing each run's output with the expected output of the test cases and awarding points per passed case. Its default local runner is for demonstration only and is not a secure sandbox, so student code must not be run through it on a shared server. The Piston connection is an untested sketch that has to be tried against a real server first.

Python is the only supported language. Grading by matching output with test cases cannot judge style, efficiency beyond the time limit or partly correct reasoning, and a weak set of test cases can pass wrong code or fail correct code. The feature does not check for copied code. The sandbox engine has to be hosted and kept up to date by the college, which adds server load and maintenance.

Feature 11: External API & Data Architecture

Core infrastructure. This feature covers how the checker models are exposed to outside systems and to AGORA itself, and how assessment data is stored internally. Together they determine how a college with its own Virtual Learning Environment (VLE) can adopt AGORA, and how reliably results are retrieved and aggregated for the E-Classrecord and analytics.

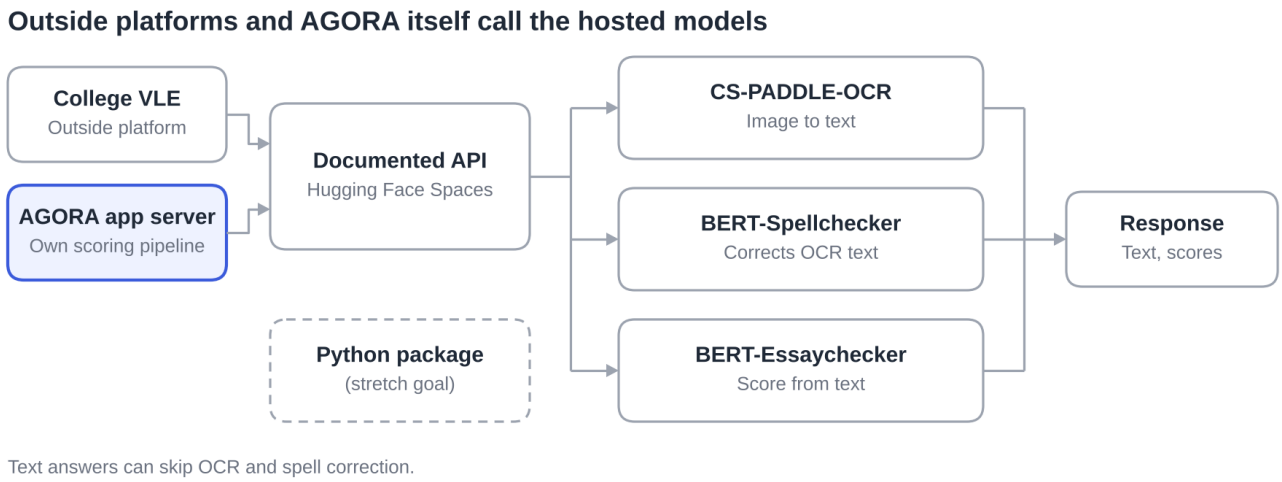


Figure 14. How an external platform and AGORA itself use the hosted models.

Core functions

AGORA exposes its three checker models as a hosted service and keeps all assessment data in one relational data layer.

Model API

- **Three models, one service.** CS-PADDLE-OCR, BERT-Spellchecker and BERT-Essaychecker are exposed as a separate, callable service. A college's existing platform can integrate with it instead of migrating to a full replacement, and AGORA's own scoring pipeline calls the same service, so there is one hosted copy of each model.
- **Hosted on Hugging Face Spaces,** a free hosting option for machine learning models.
- **Documented API.** Other platforms send scanned images or text responses and receive extracted text, corrected text and scores back, without adopting the rest of AGORA. Outside platforms are registered as API clients with hashed keys, and their calls are logged.
- **Python package (stretch goal).** The models may also be published as a package that can be installed with pip.

Data architecture

- **Relational storage.** Submissions, scores, feedback, override records and statistical summaries are stored in one PostgreSQL database, giving a structured, queryable base for individual results and class-level insight.
- **Core records:** the academic catalog (departments, programs, courses and course outcomes), classes and enrollments (the rosters), assessments (questions, answer keys and signed links), submissions (responses, scores and AI feedback), lesson plans, console prompts and outputs, the E-Classrecord, and retention analytics.

- **Reliability.** Automatic daily backups of the PostgreSQL database cover every record, including the assessment data, and the system recovers from interruptions without losing stored assessment data.

Application stack. A React web front end, backend services in Python (FastAPI) and Node.js, and local AI inference through Ollama, with all modules reaching the same data layer.

This split keeps each layer, models behind an API, data in a relational database, and the classroom and checking features on top, independently testable and replaceable.

Technologies

- **Hugging Face Spaces** is the planned free host for the three checker models behind the documented API.
- **PostgreSQL** stores all platform and assessment records and is backed up daily.
- **Python (FastAPI) and Node.js** provide the backend services and React the web front end, all reaching the same data layer.
- **Ollama** runs the local language model (Qwen 2.5) used by the Agentic Console and the ACTS NoteBook.

System limitations

The three checker models currently exist as separate command-line programs, each with its model files stored through Git LFS. The hosted API and the pip package are not built yet, so what is described under Model API is the design. Free hosting on Hugging Face Spaces has limited computing power, so a hosted model can be slow to start or to answer when several requests arrive together, and each model file is several hundred megabytes. Because AGORA's own scoring also calls the hosted models, a slow or sleeping Space slows scoring in AGORA too, and scanned images and typed answers leave the college's computers for that Space.

System Architecture

Users reach every module through one React web app and one application server, which calls local AI, a hosted model API, a code engine, an email service, a teacher workspace for saved files and one relational database.

Users reach every module through one web app and one application server

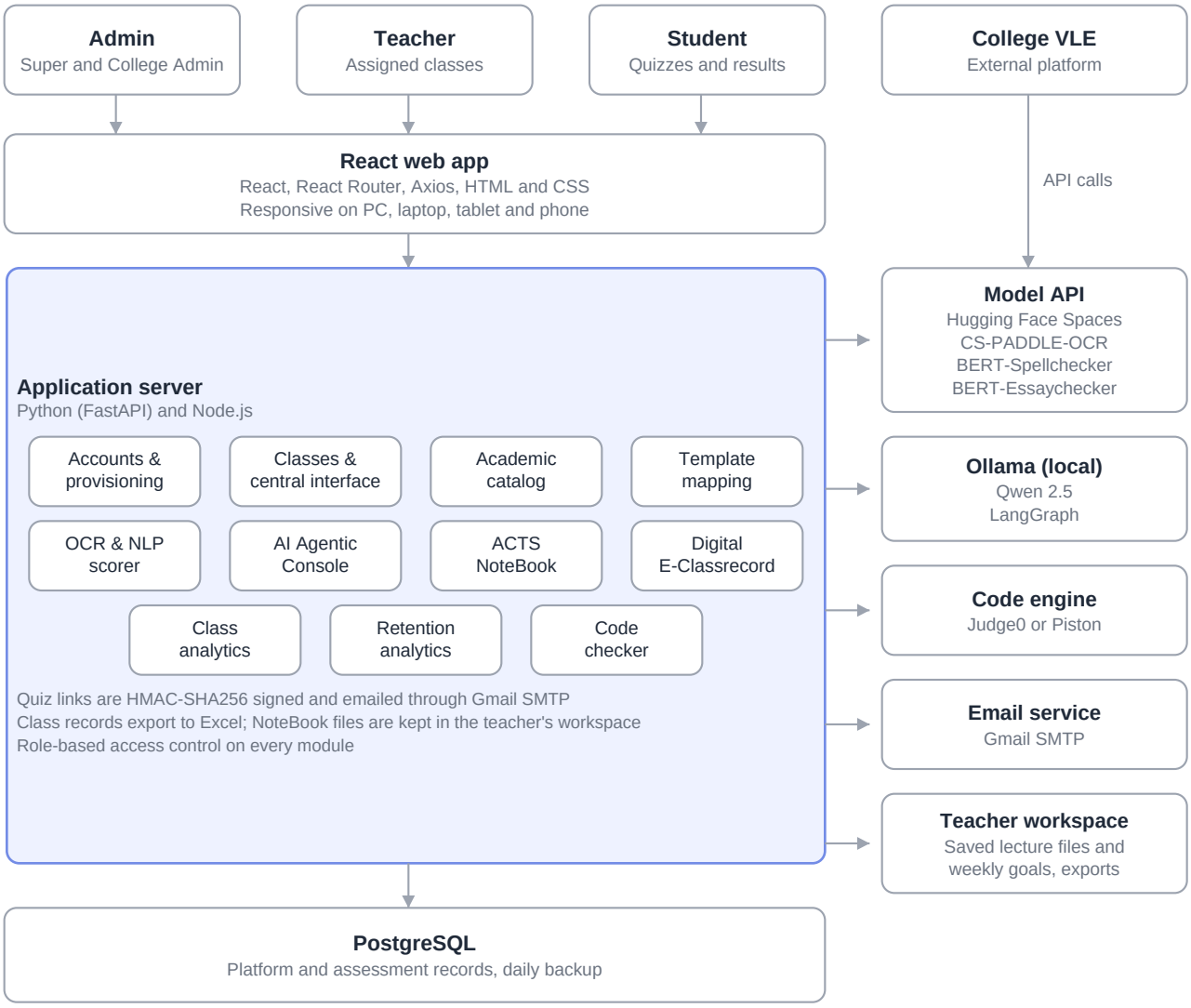


Figure 15. System architecture. External college platforms call the hosted model API directly, without using the rest of AGORA.

Database Design

The merged AGORA database adapts the original school-management schema (11 tables) to colleges. Ten of its tables are carried over (nine of them revised for college use), the `sections` table is removed, and 35 new tables are added: 29 for the AGORA features and 6 for the college structure. That makes 45 tables in total. It targets PostgreSQL and ships as `agora_schema.sql`, which keeps the original tables, revised, and adds the new enums, tables, indexes, triggers and a sample academic year with its semesters and their four grading terms. The migration script and its integrity tests are to be re-run against this college revision, together with the columns added for the updated Agentic Console, NoteBook and Retention Analytics (listed in the tables below).

Main design decisions

- **The classroom is the existing classes table,** now a course offered in one semester by one teacher, with a section code, schedule and room. Templates, assessments, grade components, attendance and analytics all reference it, so access can be scoped to the assigned teacher.
- **Enrollment is per course.** A student is linked to each class they take through enrollments, with a status such as enrolled, dropped, withdrawn or completed. This replaces the fixed section roster, and the sections table is removed.
- **The academic catalog is data.** Departments, programs, courses and course outcomes are tables, so each college keeps its own catalog, and a semester sits between the academic year and the four grading terms.
- **One database.** Platform records and assessment records live together in one PostgreSQL database with daily backups, so there is no second database engine to run, join across or keep in sync.
- **The review trail is kept.** An answer stores the raw OCR text, the corrected text and the teacher-confirmed text, plus the predicted and final score, so teacher overrides are visible and the model output is never lost. An AI essay score carries a review status and stays pending until the teacher confirms it. The per-criterion results of both essay scorers, BERT-Essaychecker and Qwen 2.5, are stored side by side and labeled by scorer.
- **Integrity is enforced by the database.** Composite foreign keys make sure an answer's question and submission belong to the same assessment, and a college can have only one current semester.
- **E-Classrecord scores are auditable.** Each score records its source (AI-synced, OCR upload, manual or override) and keeps the original AI score when a teacher overrides it. Essay scores reach the grid only after the teacher confirms them.
- **Weights, scales and thresholds are data, not code.** Grade component weights are stored per class and term, and the weight of each term in the Overall Grade is stored per class (the application checks that each set adds up to 100). Each assessment stores its essay-scoring weights and similarity cutoffs, and each answer keeps the values actually used, so older scores can be reproduced. Each college stores its own grading scale and retention baselines.
- **Secrets and files stay out of the tables.** Invite and assessment-link tokens are stored as hashes. Lecture files and weekly goals are saved in the teacher's own workspace, and the tables hold only references to them.

The tables are grouped into seven diagrams. Each group's tables are listed below, and the entity relationship diagrams follow, one per page.

Identity and provisioning (carried-over tables, plus account invites and bulk-upload log)

Table	Status	Purpose
colleges	Revised	Participating colleges and universities, added by the Super Admin. Renamed from schools, with college_code and college_type.
users	Revised	Login accounts for all four roles. school_id becomes college_id. The column must_change_password supports the planned forced reset on first login.
super_admins	Unchanged	Role profile of the Super/Master Admin, keyed by user_id.
college_admins, teachers, students	Revised	Role profiles keyed by user_id. A composite key guarantees each profile belongs to a user with that role. school_admins is now college_admins; teachers gain department_id and academic_rank; students gain program_id and year_level.
account_invites	New	Short-lived, single-use invite links, stored as hashes (Feature 1).
bulk_uploads	New	Log of each College Admin bulk upload of teachers or students: the upload type, rows created, rows failed and the errors (Feature 1). Renamed from teacher_bulk_uploads.

College setup and classes (carried-over tables, plus the academic structure)

Table	Status	Purpose
academic_years	Revised	Academic years of a college, such as 2026-2027. Renamed from school_years. The current flag moves to semesters.
semesters	New (college)	A term inside an academic year: first semester, second semester or summer, with dates. A college can have only one current semester.
departments	New (college)	Academic departments of a college. Teachers belong to a department and each course is offered by one.
programs	New (college)	Degree programs offered by a department, with degree level and years to complete. Each student belongs to one program.
courses	Revised	The course catalog: code, title, units and type (lecture, laboratory or both). Replaces subjects.
course_outcomes	New (college)	Learning outcomes of a course (for example CLO1) with a target Bloom's level. They can be loaded as the learning goals of the ACTS NoteBook (Feature 6).
classes	Revised	A course offered in one semester and taught by one teacher, with a section code, schedule, room and maximum number of students. This is the classroom of Feature 2, and everything below hangs off it.
enrollments	Revised	A student's enrollment in a class, with its status. This is the class roster, and it replaces the section-based enrollment.

Templates, assessments and questions

Table	Status	Purpose
answer_sheet_templates	New	Reusable sheet layout saved in a classroom (Feature 3). It can be duplicated to the teacher's other classes for the same course.
template_regions	New	One region of the sheet: question type, table or cells layout, and the aligned matrix geometry.
template_answer_keys	New	Answer key per item of a region, for objective checking without a model.
assessments	New	A quiz or exam for a class: delivery (digital or scanned), status (for example draft, approved or sent), deadline, allowed attempts, and the essay-scoring weights (mechanics and content) and similarity cutoffs (full and partial) used for its essay questions (Features 4, 5).
questions	New	Items of an assessment with type, prompt, choices, answer key and points.
rubric_criteria	New	Teacher-written rubric criteria and point values for essay and short-answer questions (Feature 4).
coding_problems	New	A coding question's statement, language and time limit (Feature 10).
code_test_cases	New	Input and expected-output pairs used to grade code.
assessment_links	New	Per-student HMAC-SHA256 secured link, stored as a token hash with expiry and sent time (Feature 5).

Submissions and scoring

Table	Status	Purpose
submissions	New	One attempt by a student: digital, scanned or typed by the teacher, with status, total score and AI feedback.
answers	New	Per-question answer. Keeps raw OCR text, spell-corrected text, teacher-confirmed final text, mechanics and content sub-scores, predicted and final score, the scoring weights and cutoffs actually used, low-confidence flag and reviewer. Also keeps a **review status**, so an AI essay score stays pending until the teacher confirms it, and the **time of confirmation** (Feature 5).
rubric_results	New	Per-criterion rubric match: the scorer that produced it (BERT-Essaychecker or Qwen 2.5), best sentence, similarity, verdict (met, partial, not met) and points awarded. Also keeps a short **reason** and the AI's **confidence** for each criterion (Feature 5).
code_runs	New	Each code run: status, stdout, stderr, run time, passed test cases and the result of each test case.

Grades

Table	Status	Purpose
grading_terms	New	Prelim, Midterm, Pre-Final and Final for a semester (Feature 7).
class_term_weights	New	The weight of each grading term in a class's Overall Grade (25% each by default, adjustable; the application checks they add up to 100) (Feature 7).
grade_components	New	Weighted components per class and term, such as Written Work and Performance Task.
eclass_columns	New	Assessment columns of the grid, ordered by drag and drop and optionally linked to an assessment.
eclass_scores	New	A student's score in a column, with its source (AI-synced, manual, override or OCR upload) and the original AI score for audit. Quiz scores that include essays are written only after the teacher confirms them.
grade_scale_bands	New (college)	The college's grading scale: percentage ranges mapped to a grade label and grade point (for example 1.00 to 5.00 or letter grades), with passing bands marked. Used to convert the Overall Grade (Feature 7).

Attendance and analytics

Table	Status	Purpose
attendance_records	New	Daily attendance per class, the second input of retention analytics (Feature 9).
retention_settings	New (college)	Per-college baselines for grades and attendance (75% by default) read by the label logic (Feature 9).
retention_analytics	New	Per student and class: running average, attendance rate, quadrant, risk score and intervention. Also keeps the **label** (Safe, For Counseling or At Risk); the quadrant shows whether a For Counseling label is attendance (yellow) or academic (orange) (Feature 9).
class_performance_snapshots	New	Stored statistics per class or assessment: mean, median, spread, extremes and per-item miss rates (Feature 8).

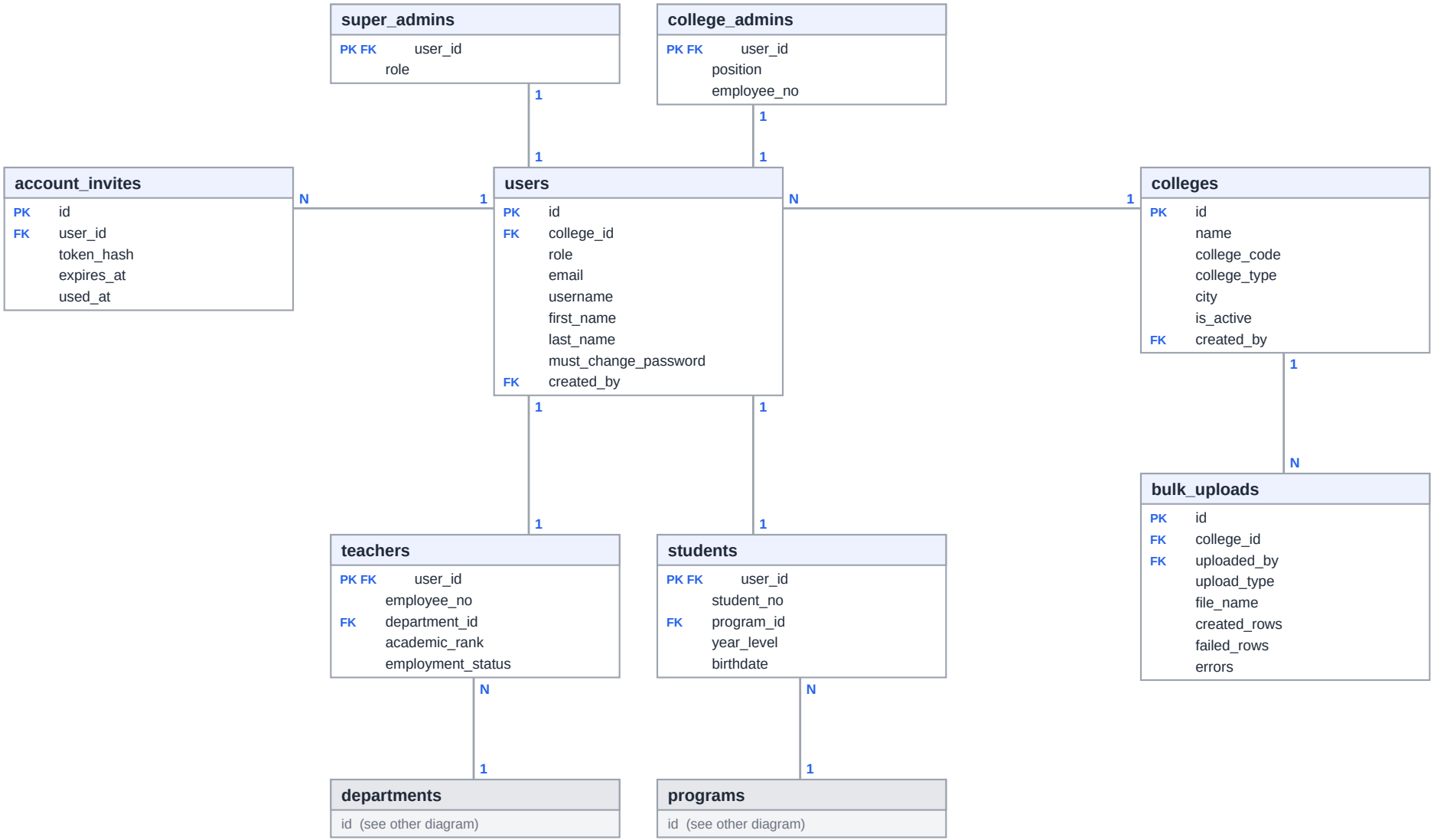
AI services and platform

Table	Status	Purpose
lesson_plans	New	ACTS NoteBook output: topics, the **weekly goals**, references to the lecture files saved in the teacher's workspace, the **character count extracted from each file**, generated OBE content and export links (Feature 6).
teacher_obe_preferences	New	Saved OBE preferences looked up by the Preference Collector step.
agent_prompts	New	Agentic Console runs: the **task type** (create quiz, send quiz or retrieve class information), the teacher's request, model settings, agent outputs and status (Feature 5).
api_clients	New	Outside platforms allowed to call the hosted model API, with hashed keys (Feature 11).
api_requests	New	Usage log of API calls per client.
notifications	New	Central Interface notifications per user (Feature 2).

Changes to the original schema. The 11 original tables are carried over except sections, which is removed. schools becomes colleges, school_admins becomes college_admins, school_years becomes academic_years and subjects becomes courses, and every school_id column becomes college_id. teachers gain department_id and academic_rank. students gain program_id and year_level, and no longer store a guardian. classes now means a course offered in a semester (it gains semester_id, course_id and section_code, and loses section_id). enrollments now link a student to a class instead of to a section. users keeps the must_change_password column added earlier. Status values: Unchanged means an original table carried over as is; Revised means an original table adapted for colleges; New means added for the AGORA features; New (college) means added for the college structure. Bold text in the tables above marks columns added for the updated Agentic Console, NoteBook and Retention Analytics: answers.review_status and confirmed_at; rubric_results.reason and confidence; retention_analytics.label; lesson_plans.weekly_goals and source_char_counts; agent_prompts.task_type.

Aligned with the feature descriptions. To match what Features 1, 4, 5, 7 and 10 say is stored, the schema also has: class_term_weights; max_attempts and the essay-scoring weights and cutoffs on assessments; scoring_params on answers; choices and answer_key on questions; scorer on rubric_results; upload_type and errors on bulk_uploads (renamed from teacher_bulk_uploads, since it now logs student uploads too); and case_results on code_runs.

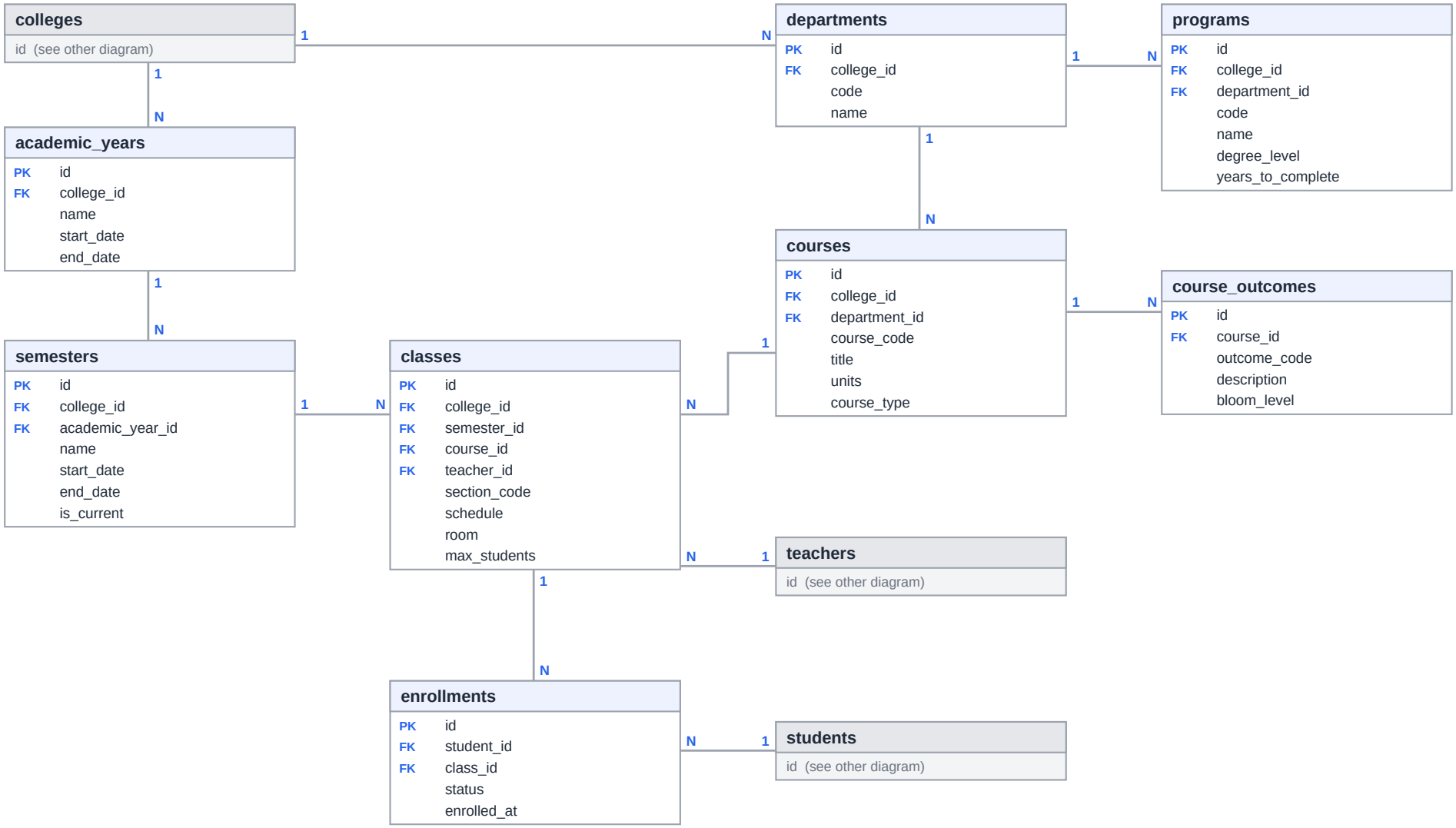
Entity relationship diagram 1: Identity and provisioning



users.created_by and colleges.created_by also point back to users (self-references), and bulk_uploads.uploaded_by points to users (not drawn).
Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 16. Entity relationship diagram: identity and provisioning. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

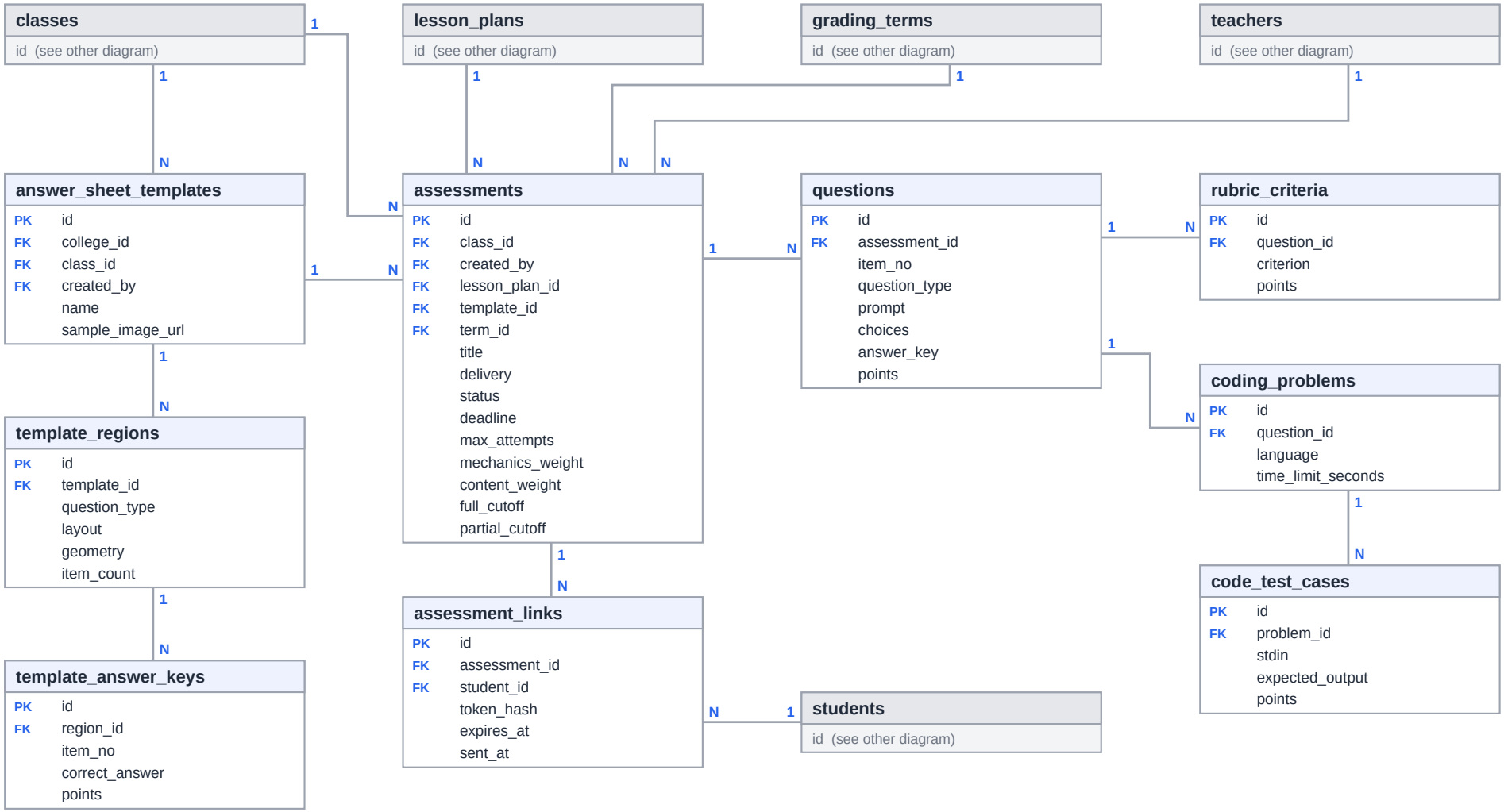
Entity relationship diagram 2: College setup and classes



college_id on semesters, programs, courses and classes also references colleges (not drawn). teachers.department_id and students.program_id are drawn in diagram 1. Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 17. Entity relationship diagram: college setup and classes. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

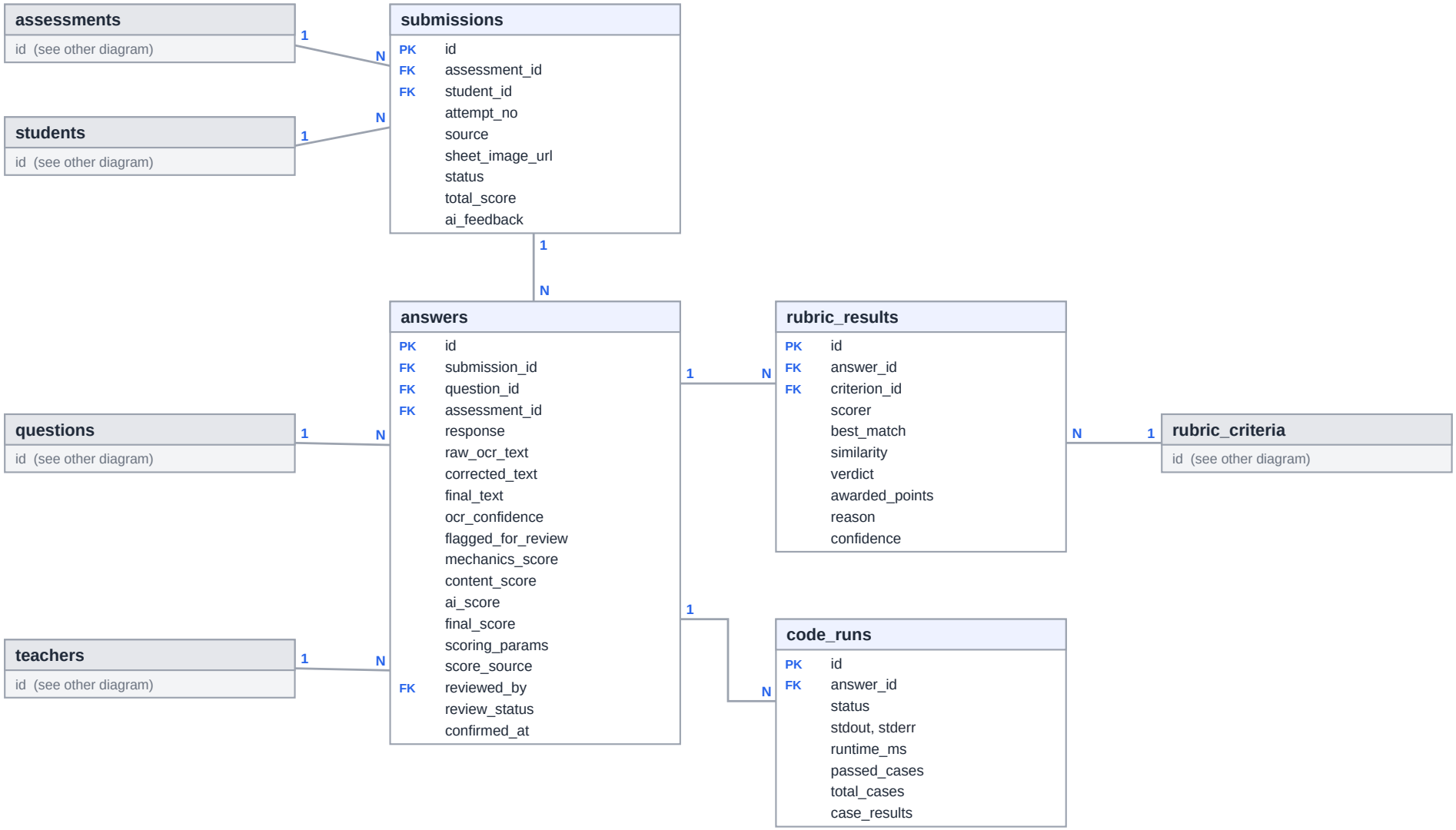
Entity relationship diagram 3: Templates, assessments and questions



Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 18. Entity relationship diagram: templates, assessments and questions. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

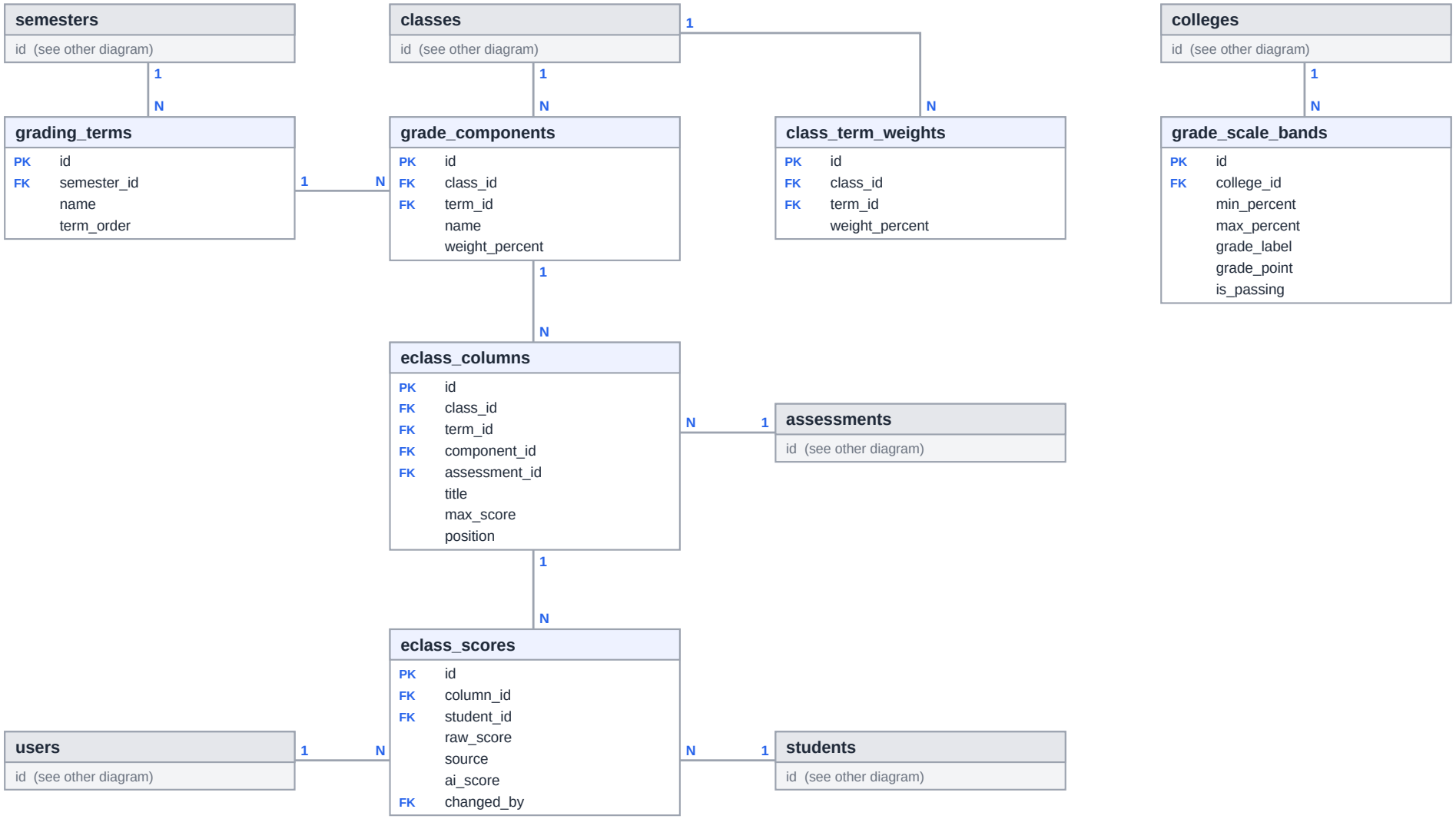
Entity relationship diagram 4: Submissions and scoring



Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 19. Entity relationship diagram: submissions and scoring. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

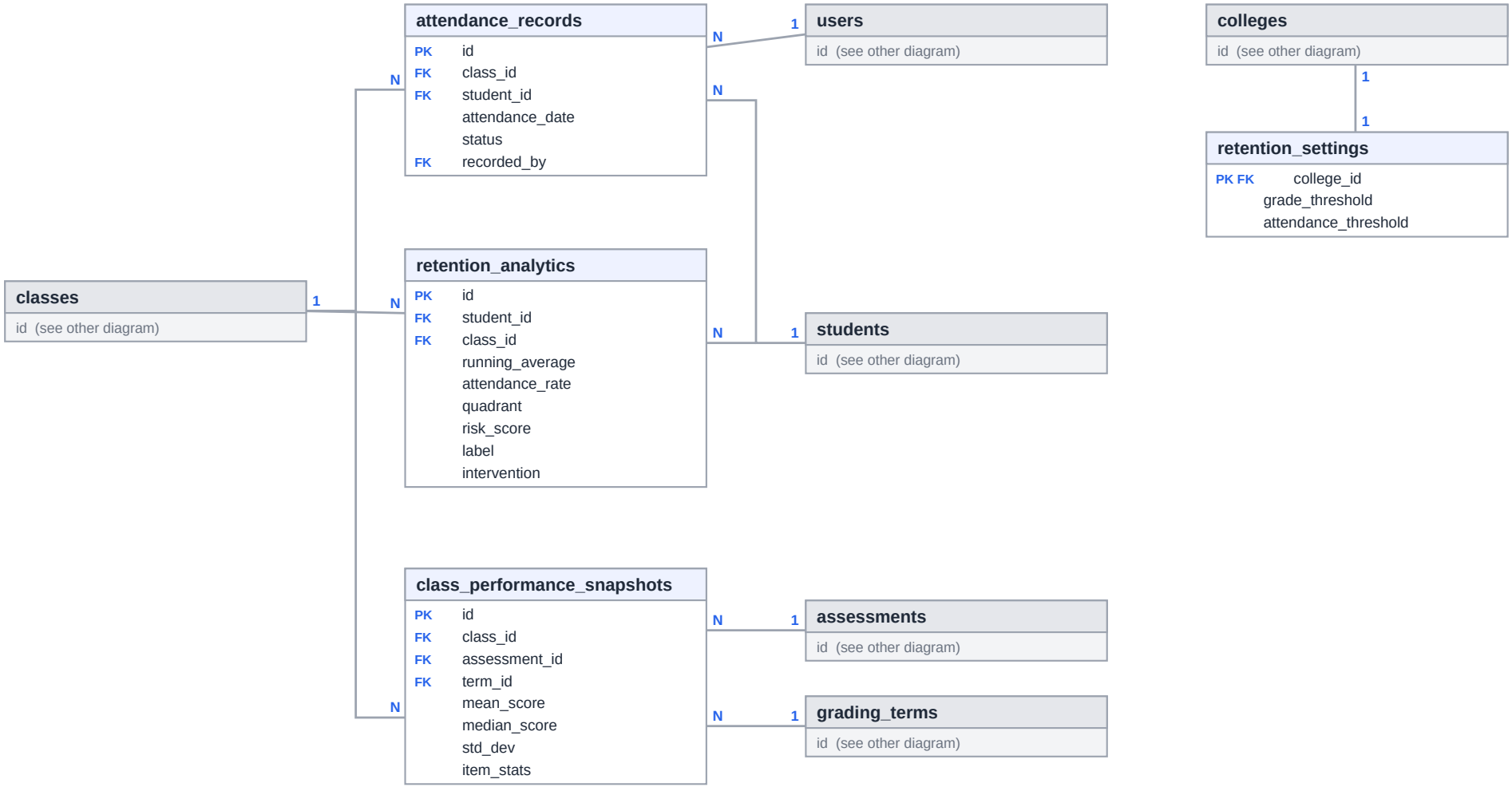
Entity relationship diagram 5: Grades



The grading scale is looked up when an Overall Grade is converted (for example to 1.00 to 5.00); it is not linked to individual scores.
class_term_weights and eclass_columns also carry term_id (grading_terms), and eclass_columns also carries class_id (classes); these links are not drawn.
Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 20. Entity relationship diagram: grades. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

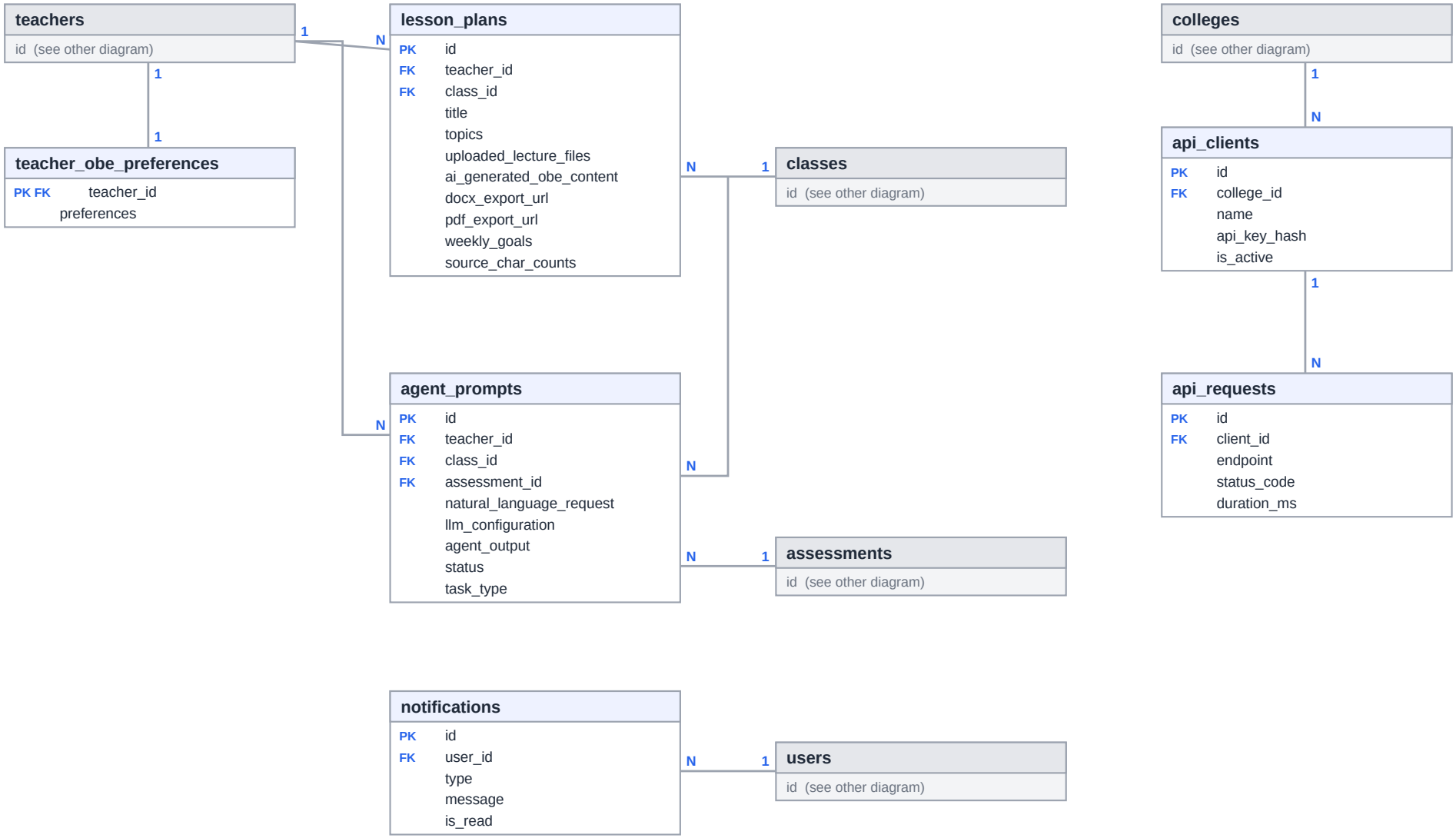
Entity relationship diagram 6: Attendance and analytics



retention_settings holds the baselines (75% by default) that the quadrant classifier reads; it is looked up, not linked to each row.
Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 21. Entity relationship diagram: attendance and analytics. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.

Entity relationship diagram 7: AI services and platform



Diagrams show the main columns of each table; agora_schema.sql is the full definition. One-to-one links (primary key that is also a foreign key) are drawn 1 to 1.

Figure 22. Entity relationship diagram: AI services and platform. N = many, 1 = one. Grey boxes are tables shown in full in another diagram.